

Modern C++ for Scientific Computing

Victor Eijkhout

TACCster 2024

Introduction

1. Write modern C++

1. C++ is a more structured and safer variant of C:
There are very few reasons not to switch to C++.
2. C++ has undergone many changes in the last dozen years
Many mechanisms make programming easier and safer
(not faster: you'll never beat writing C)

2. In this course

1. Minor enhancements
2. Looping and ranges
3. Smart pointers
4. Multi-dimensional arrays
5. OpenMP mechanisms
6. A nice looking MPI library. Maybe.
7. More.

I'm assuming that you know how to code C loops and functions and you understand what structures and pointers are!

3. About this course

Slides and codes are from my open source text book:

<https://theartofhpc.com/isp.html>

4. General note about standards

Many of the examples in this lecture use the C++17/20/23 (sometimes C++26) standard.

```
icpc      -std=c++20 yourprogram.cxx  
icpx      -std=c++20 yourprogram.cxx  
g++       -std=c++20 yourprogram.cxx  
clang++   -std=c++20 yourprogram.cxx
```

There is no reason not to use that all the time:

```
alias icpc='icpc -std=c++20'  
et cetera
```

5. Build with Cmake

```
cmake_minimum_required( VERSION 3.20 )
project( ${PROGRAM_NAME} VERSION 1.0 )

add_executable( ${PROGRAM_NAME} ${PROGRAM_NAME}.cpp )
target_compile_features( ${PROGRAM_NAME}
    PRIVATE cxx_std_20 )
```

6. C++ standard

- C++98/C++03: ancient.
There was a lot wrong or not-great with this.
- C++11/14/17: 'modern' C++.
What everyone uses.
- C++20/23: 'post-modern' C++.
Ratified, but only partly implemented.
- C++26: 'feature freeze' in 2025; some features already available.

Minor enhancements

7. Initializer statement

Loop variable can be local (also in C99):

```
for (int i=0; i<N; i++) // do whatever
```

Similar in conditionals and switch:

```
// basic/ifinit.cpp
if ( char c = getchar(); c!='a' )
    cout << "Not an a, but: "
         << c << '\n';
else
    cout << "That was an a!"
         << '\n';
```

(strangely not in `while`)

8. Simple I/O

Headers:

```
#include <iostream>
using std::cin;
using std::cout;
```

Output:

```
int main() {
    int plan=4;
    cout << "Plan " << plan << " from outer space" << "\n";
```

9. Format library

printf-ish formatting:

```
#include <format>

string std::format( /* stuff */ );
void    std::print ( /* stuff */ ); // as of C++23
```

(also available as external `fmtlib`)

10. Simple example

The basic usage is:

```
int i=2;  
format("string {} brace expressions",i);
```

Format string, and arguments.

11. Right align

Right-align with > character and width:

Code:

```
// io/fmtlib.cpp
for (int i=10; i<2000000000;
    i*=10)
    fmt::print("{:>6}\n", i);
```

Output:

```
    10
   100
  1000
 10000
100000
1000000
10000000
100000000
```

12. Padding character

Other than space for padding:

Code:

```
// io/fmtlib.cpp
for (int i=10; i<2000000000;
     i*=10)
    fmt::print("{0:.>6}\n",i);
```

Output:

```
....10
...100
..1000
.10000
100000
1000000
10000000
100000000
```

13. Number bases

Code:

```
// io/fmtlib.cpp
fmt::print
    ("{0} = {0:b} bin\n",17);
fmt::print
    ("    = {0:o} oct\n",17);
fmt::print
    ("    = {0:x} hex\n",17);
```

Output:

```
17 = 10001 bin
    = 21 oct
    = 11 hex
```


14. Float and fixed

Floating point or normalized exponential with *e* specifier

fixed: use decimal point if it fits, *m.n* specification

Code:

```
// iofmt/fmtfloat.cpp
x = 1.234567;
for (int i=0; i<6; ++i) {
    cout <<

        format("{0:.3e}/{0:7.4}\n",
                x);
    x *= 10;
}
```

Output:

```
1.235e+00/  1.235
1.235e+01/ 12.35
1.235e+02/ 123.5
1.235e+03/ 1235
1.235e+04/1.235e+04
1.235e+05/1.235e+05
```

Functions

15. Parameter passing by reference

The function parameter *n* becomes a reference to the variable *i* in the main program:

```
1 void f(int &n) {  
2     n = /* some expression */ ;  
3 };  
4 int main() {  
5     int i;  
6     f(i);  
7     // i now has the value that was set in the function  
8 }
```

16. Const ref parameters

```
void f( const int &i ) { .... }
```

- Pass by reference: no copying, so cheap
- Const: no accidental altering.
- Especially useful for large objects.

17. Default arguments

Functions can have default argument(s):

```
double distance( double x, double y=0. ) {  
    return sqrt( (x-y)*(x-y) );  
}  
  
...  
d = distance(x); // distance to origin  
d = distance(x,y); // distance between two points
```

Any default argument(s) should come last in the parameter list.

18. Useful idiom

Don't trace a function unless I say so:

```
void dosomething(double x, bool trace=false) {  
    if (trace) // report on stuff  
};  
  
int main() {  
    dosomething(1); // this one I trust  
    dosomething(2); // this one I trust  
    dosomething(3, true); // this one I want to trace!  
    dosomething(4); // this one I trust  
    dosomething(5); // this one I trust
```

19. Polymorphic functions

You can have multiple functions with the same name:

```
double average(double a, double b) {  
    return (a+b)/2; }  
double average(double a, double b, double c) {  
    return (a+b+c)/3; }
```

Distinguished by type or number of input arguments: can not differ only in return type.

```
int f(int x);  
string f(int x); // DOES NOT WORK
```

Lambda expressions

20. Introducing: lambda expressions

Traditional function usage:

explicitly define a function and apply it:

```
float sum(float x,float y) { return x+y; }  
cout << sum( 1.2f, 3.4f );
```

New:

apply the function recipe directly:

Code:

```
// lambda/lambdaex.cpp  
[] (float x,float y) -> float {  
    return x+y; } ( 1.5, 2.3 )
```

Output:

3.8

21. Lambda syntax

```
[capture] ( inputs ) -> outtype { definition };  
[capture] ( inputs ) { definition };
```

- The square brackets are how you recognize a lambda; we will get to the 'capture' later. For now it will often be empty.
- Inputs: like function parameters
- Result type specification `-> outtype`: can be omitted if compiler can deduce it;
- Definition: function body.

22. Lambdas as parameter: the problem

Lambdas have a type that is dynamically generated, so you can not write a function that takes a lambda as argument, because you can't write the type.

```
1 void apply_to_5( /* what? */ func ) {  
2     func(5);  
3 }  
4 int main() {  
5     apply_to_5  
6     ( [] (double x) { cout << x; } );  
7 }
```

23. Lambdas as parameter: the solution

Header:

```
#include <functional>
using std::function;
```

declare function parameters by their signature
(that is, types of parameters and output):

Code:

```
// lambda/lambdaex.cpp
void apply_to_5
( function< void(int) > f
) {
    f(5);
}

/* ... */
apply_to_5
( [] (int i) {
    cout << "Int: " << i
        << '\n'; } );
```

Output:

Int: 5

24. Newton's method

$$x = \sqrt{y} \Leftrightarrow f_y(x) = x^2 - y = 0$$

```
double newton_root                                     }
( function<                                             return x;
  double(double) > f,                                   };
  function<
  double(double) > fprime
) {
double x{1.};
while ( true ) {
  auto fx = f(x);
  if (std::abs(fx)<1.e-10 )
    break;
  x = x - fx/fprime(x);
  cout << "sqrt 2 = " <<
    newton_root(
      [] (double x) { return
        x*x-2; },
      [] (double x) { return
        2*x; } )
    << '\n';
}
```

25. Capture variable

Increment function:

- scalar in, scalar out;
- the increment amount has been fixed through the capture.

Code:

```
// lambda/lambdacapture.cpp
int n;
cin >> n;
auto increment_by_n =
    [n] ( int input ) -> int {
        return input+n;
    };
cout << increment_by_n (5) << '\n';
cout << increment_by_n (12) << '\n';
cout << increment_by_n (25) << '\n';
```

Output:

(input value:
↪1)

6
13
26

26. Capture value is copied

Illustrating that the capture variable is copied once and for all:

Code:

```
// lambda/lambdacapture.cpp
int inc;
cin >> inc;
auto increment =
    [inc] ( int input ) -> int {
        return input+inc;
    };
cout << "increment by: "
      << inc << '\n';
cout << "1 -> "
      << increment(1) << '\n';
inc = 2*inc;
cout << "1 -> "
      << increment(1) << '\n';
```

Output:

```
increment by: 2
1 -> 3
1 -> 3
```

27. Newton with capture

```
for (int n=2; n<=8; ++n) {  
    cout << "sqrt(" << n << ") = "  
        << newton_root(  
            [n] (double x) { return x*x-n; },  
            [] (double x) { return 2*x; }  
        )  
        << '\n';  
}
```


28. Gradient by finite differences

```
double newton_root( function< double(double) > f, double
    h=.001 ) {
    cout << "gradient-free newton with h=" << h << '\n';
    return newton_root( f, [f,h] (double x) { return
        (f(x+h)-f(x))/h; } );
};
```

Vectors, loops, ranges

29. Short vectors

Short vectors can be created by enumerating their elements:

```
1 // array/shortvector.cpp
2 #include <vector>
3 using std::vector;
4
5 int main() {
6     vector<int> evens{0,2,4,6,8};
7     vector<float> halves = {0.5, 1.5, 2.5};
8     auto halffloats = {0.5f, 1.5f, 2.5f};
9     cout << evens.at(0)
10          << " from " << evens.size()
11          << '\n';
12     return 0;
13 }
```

30. Range over elements

A range-based for loop gives you directly the element values:

```
vector<float> my_data(N);  
/* set the elements somehow */;  
for ( float e : my_data )  
    // statement about element e
```

Here there are no indices because you don't need them.

31. Range over elements, version 2

Same with auto instead of an explicit type for the elements:

```
for ( auto e : my_data )  
    // same, with type deduced by compiler
```

32. Range over elements by reference

Range-based loop indexing makes a copy of the vector element. If you want to alter the vector, use a reference:

```
for ( auto &e : my_vector )  
    e = ....
```

Code:

```
// array/vectorrangeref.cpp  
vector<float> myvector  
    = {1.1, 2.2, 3.3};  
for ( auto &e : myvector )  
    e *= 2;  
cout << myvector.at(2)  
      << '\n';
```

Output:

6.6

(Can also use `const auto& e` to prevent copying, but also prevent altering data.)

33. Range over vector denotation

Code:

```
// array/rangedenote.cpp  
for ( auto i : {2,3,5,7,9} )  
    cout << i << ",";  
cout << '\n';
```

Output:

2,3,5,7,9,

Exercise 1

Create a vector x of `float` elements, and set them to random values. (Use the C random number generator for now.)

Now normalize the vector in L_2 norm and check the correctness of your calculation, that is,

1. Compute the L_2 norm of the vector:

$$\|v\| \equiv \sqrt{\sum_i v_i^2}$$

2. Divide each element by that norm;
3. The norm of the scaled vector should now be 1. Check this.
4. Bonus: your program may be printing 1, but is it actually 1? Investigate.

What type of loop are you using?

34. Range-based iteration

You have seen

```
for ( auto n : set_of_integers )  
    if ( even(n) )  
        do_something(n);
```

Can we do

```
for ( auto n : set_of_integers  
    and even ) // <= not actual syntax  
    do_something(n);
```

or even

```
// again, not actual syntax  
apply( set_of_integers and even,  
    do_something );
```

35. Range algorithms

Algorithms: for-each, find, filter, transform ...

Ranges: iterable things such as vectors

Views: transformations of ranges, such as picking only even numbers

C++20 ranges

36. Range over vector

With

```
// rangestd/range.cpp
vector<int> generate_data() { return {2,3,4,5,6,7}; };
/* ... */
auto v = generate_data();
```

Code:

```
// rangestd/range.cpp
#include <algorithm>
#include <ranges>
namespace rng = std::ranges;
/* ... */
rng::for_each
( v,
  [] (int i) {
    cout << i << " ";
  }
);
```

Output:

2 3 4 5 6 7

37. Ranged algorithm

With

```
// rangestd/range.cpp
vector<int> generate_data() { return {2,3,4,5,6,7}; };
/* ... */
auto v = generate_data();
```

Code:

```
// rangestd/range.cpp
count = 0;
rng::for_each
( v,
  [&count] (int i) {
    count += (i<5); }
);
cout << "Under five: "
      << count << '\n';
```

Output:

Under five: 3

38. Range composition

Pipeline of ranges and views:

```
// rangestd/range.cpp
vector<int> generate_data() { return {2,3,4,5,6,7}; };
/* ... */
auto v = generate_data();
```

Code:

```
// rangestd/range.cpp
count = 0;
rng::for_each
( v | rng::views::drop(1),
  [&count] (int i) {
    count += (i<5); }
);
cout << "minus first: "
      << count << '\n';
```

Output:

minus first: 2

39. Iota and take

Code:

```
// rangestd/iota.cpp
#include <ranges>
namespace rng = std::ranges;
    /* ... */
    for ( auto n :
        rng::views::iota(2,6) )
        cout << n << '\n';
    cout << "===\n";
    for ( auto n :
        rng::views::iota(2)
        | rng::views::take(4) )
        cout << n << '\n';
```

Output:

```
2
3
4
5
===
2
3
4
5
```

40. Filter

Take a range, and make a new one of only the elements satisfying some condition:

Code:

```
// rangestd/filter.cpp
vector<float> numbers
{1,-2.2,3.3,-5,7.7,-10};
auto pos_view =
    numbers
    | std::ranges::views::filter
      ( [] (int i) -> bool {
          return i>0; }
      );
for ( auto n : pos_view )
    cout << n << " ";
cout << '\n';
```

Output:

```
1 3.3 7.7
```


Exercise 2: Element counting

Change the filter example to let the lambda count how many elements were > 0 .

41. Range composition

Code:

```
// range/filtertransform.cpp
vector<int> v{ 1,2,3,4,5,6 };
/* ... */
auto times_two_over_five = v
    | rng::views::transform
      ( [] (int i) {
          return 2*i; } )
    | rng::views::filter
      ( [] (int i) {
          return i>5; } );
```

Output:

Original vector:
1, 2, 3, 4, 5, 6,
Times two over five:
6 8 10 12

42. Quantor-like algorithms

Code:

```
// rangestd/of.cpp
vector<int>
    integers{1,2,3,5,7,10};
auto any_even =
    std::ranges::any_of
        ( integers,
          [=] (int i) -> bool {
              return i%2==0; }
        );
if (any_even)
    cout << "there was an
            even\n";
else
    cout << "none were even\n";
```

Output:

there was an even

Also *all_of*, *none_of*

43. Reductions

accumulate and *reduce*:
tricky, and not in all compilers.
See above for an alternative.

Exercise 3: Perfect numbers

A perfect number is the sum of its own divisors:

$$6 = 1 + 2 + 3$$

Output the perfect numbers.

(at least 4 of them)

Use only ranges and algorithms, no explicit loops.

44. Why ranges?

- Loops without bounds problems.
- Parallelism:

```
for_each( std::execution::par_unseq, range, alg );
```

(Automatically OpenMP and GPU)

Multi-dimensional arrays

45. Using subarrays

Form *subarray* as part of *array* that starts at the second element:

```
double *array = new double[N];  
double *subarray = array+1;  
subarray[1] = 5.; // same as: array[2] = 5.;
```

Using ‘subarrays’ would be useful, for instance in a quicksort algorithm:

```
// Warning: this is pseudo-code  
void qs( data ) {  
    if (data.size()>1) {  
        // pivoting stuff omitted  
        qs( data.lefthalf() ); qs( data.lefthalf() );  
    }  
}
```


46. Span

Create a `span` from a `vector`, starting at its second element:

```
#include <span>
vector<double> v;
std::span<double> v_span( v.data()+1, v.size()-1 );
```

47. mdspan

Header: `mdspan`

Create 2D `mdspan` from vector:

```
// mdspan/index2.cpp
// matrix in row major
vector<float> A(M*N);
md::mdspan
    Amd{ A.data(),md::extents{M,N} };
```

48. Four-d mdspan matrix

Construct a multi-dimensional span from a vector:

```
vector<float> ar10203040(10*20*30*40);  
auto brick10203040 =  
    std::mdspan< float, extents<10,20,30,40> >(  
        ar10203040.data() );  
auto midpoint = brick10203040[5,10,15,20];
```

49. Rowsum calculation

Given `mdspan mat`, find its sizes, extract each row, and the sum of its elements:

```
// mdspan/index2.cpp
int M = mat.extent(0); int N = mat.extent(1);
vector<float> rowsums(N);
for ( int row=0; auto& rs : rowsums ) {
    auto the_row =
        rng::iota_view(0,M)
        | rng::views::transform
        ( [mat,row] (int col) -> float {
            return mat[row,col]; } );
    rs = rng::accumulate( the_row, 0.f );
    row++;
}
```

Note that `the_row` is a view, not a data structure.

Smart pointers

50. Pointer types

- Smart pointers. You will see ‘shared pointers’.
- There are ‘unique pointers’. Those are tricky.
- Please don’t use old-style C pointers, unless you become very advanced.

51. Example: step 1, we need a class

Simple class that stores one number:

Definition:

```
// pointer/pointx.cpp
class HasX {
private:
    double x;
public:
    HasX( double x) : x(x) {};
    auto value() { return x; };
    void set(double xx) {
        x = xx; };
};
```

Example usage

```
// pointer/pointx.cpp
HasX xobj(5);
cout << xobj.value() << '\n';
xobj.set(6);
cout << xobj.value() << '\n';
```

52. Example: step 2, creating the pointer

Allocation of object and pointer to it in one:

```
auto X = make_shared<HasX>( /* args */ );
```

```
// or explicitly:
```

```
shared_ptr<HasX> X =  
    make_shared<HasX>( /* constructor args */ );
```


53. Use of a shared pointer

Object vs pointed-object:

Code:

```
// pointer/pointx.cpp
#include <memory>
using std::make_shared;

/* ... */
HasX xobj(5);
cout << xobj.value() <<
'\n';
xobj.set(6);
cout << xobj.value() <<
'\n';

auto xptr =
make_shared<HasX>(5);
cout << xptr->value() <<
'\n';

xptr->set(6);
cout << xptr->value() <<
```

Output:

5
6
5
6

54. Example: step 3: headers to include

Using smart pointers requires at the top of your file:

```
#include <memory>
using std::shared_ptr;
using std::make_shared;

using std::unique_ptr;
using std::make_unique;
```

55. Getting the underlying pointer

```
X->y;  
// is the same as  
X.get()->y;  
// is the same as  
( *X.get() ).y;
```

Code:

```
// pointer/pointy.cpp  
auto Y = make_shared<HasY>(5);  
cout << Y->y << '\n';  
Y.get()->y = 6;  
cout << ( *Y.get() ).y <<  
    '\n';
```

Output:

5
6

56. Pointers don't go with addresses

The oldstyle `&y` address pointer can not be made smart:

```
// pointer/address.cpp
auto
    p = shared_ptr<HasY>( &y );
p->y = 3;
cout << "Pointer's y: "
      << p->y << '\n';
```

gives:

```
address(56325,0x7fff977cc380) malloc: *** error for object
0x7ffeeb9caf08: pointer being freed was not allocated
```

57. Example: step 4: in use

Set two pointers to the same object:

Code:

```
// pointer/pointx.cpp
auto xptr =
    make_shared<HasX>(5);
auto yptr = xptr;
cout << xptr->value() << '\n';
yptr->set(6);
cout << xptr->value() << '\n';
```

Output:

5
6

58. Pointer dereferencing

Example: function

```
float distance_to_origin( Point p );
```

How do you apply that to a `shared_ptr<Point>`?

```
shared_ptr<Point> p;  
distance_to_origin( *p );
```

59. Null pointer

Initialize smart pointer to null pointer; test on null value:

```
shared_ptr<Foo> foo_ptr = nullptr;  
// stuff  
if (foo_ptr!=nullptr)  
    foo_ptr->do_something();
```

Stuff

60. Returning tuple with type deduction

Return type deduction:

```
1 // stl/tuple.cpp
2 auto maybe_root1(float x) {
3     if (x<0)
4         return make_tuple
5             <bool,float>(false,-1);
6     else
7         return make_tuple
8             <bool,float>
9             (true,sqrt(x));
10 };
```

Alternative:

```
1 // stl/tuple.cpp
2 tuple<bool,float>
3     maybe_root2(float x) {
4     if (x<0)
5         return {false,-1};
6     else
7         return {true,sqrt(x)};
8 };
```

Note: use *pair* for *tuple* of two.

61. Catching a returned tuple

The calling code is particularly elegant:

Code:

```
// stl/tuple.cpp
auto [succeed,y] =
    maybe_root2(x);
if (succeed)
    cout << "Root of " << x
          << " is " << y << '\n';
else
    cout << "Sorry, " << x
          << " is negative" <<
          '\n';
```

Output:

```
Root of 2 is 1.41421
Sorry, -2 is negative
```

This is known as structured binding.

62. Create optional

```
optional<float> f {  
    if (something)  
        // result if success  
        return 3.14;  
    else  
        // indicate failure  
        return {};  
}
```

63. Testing and getting value

Two ways:

```
// union/optroot.cpp
for ( auto x : {2.f,-2.f} )
    if ( auto root =
        MaybeRoot(x) ;
        root.has_value() )
        cout << "Root is " <<
            root.value() << '\n';
else
    cout << "could not take
        root of " << x << '\n';
```

```
// union/optroot.cpp
for ( auto x : {2.f,-2.f} )
    if ( auto root =
        MaybeRoot(x) ; root )
        cout << "Root is " <<
            *root << '\n';
    else
        cout << "could not take
            root of " << x << '\n';
```

64. Expected

Expect double, return info string if not:

```
std::expected<double,string>
    square_root( double x
    ) {
    auto result = sqrt(x);
    if (x<0)
    return
        std::unexpected("negative"); /* handle the problem */
    else if
        (x<limits<double>::min())
    return
        std::unexpected("underflow");
    else return result;
}
```

```
auto root = square_root(x);
if (x)
    cout << "Root=" <<
        root.value() << '\n';
else if (root.error()=/* et
        cetera */ )
```

Exercise 4

Write a routine that computes the roots of the quadratic equation

$$ax^2 + bx + c = 0.$$

The routine should return two roots, or one root, or an indication that the equation has no solutions.

Code:

```
// union/quadratic.cpp
for ( auto coefficients :
    { quadratic{.a=2.0,
    .b=1.5, .c=2.5},
      quadratic{.a=1.0,
    .b=4.0, .c=4.0},
      quadratic{.a=2.2,
    .b=5.1, .c=2.5}
    } ) {
    auto result =
        compute_roots(coefficients);
```

Output:

With a=2 b=1.5 c=2.5

No root

With a=2.2 b=5.1

↪ c=2.5

Root1: -0.703978

↪ root2: -1.6142

With a=1 b=4 c=4

Single root: -2

OpenMP parallel loops

Questions

1. Do regular OpenMP loops look different in C++?
2. Is there a relation between OpenMP parallel loops and iterators?
3. OpenMP parallel loops vs parallel execution policies on algorithms.

Range syntax

Parallel loops in C++ can use range-based syntax as of OpenMP-5.0:

```
// vecdata.cxx
vector<float> values(100);

#pragma omp parallel for
for ( auto& elt : values ) {
    elt = 5.f;
}

float sum{0.f};
#pragma omp parallel for reduction(+:sum)
for ( auto elt : values ) {
    sum += elt;
}
```

Tests show exactly the same speedup as the C code.

General idea

OpenMP can parallelize any loop over a C++ construct that has a 'random-access' iterator.

C++ ranges header

The C++20 ranges library is supported:

```
#pragma omp parallel for reduction(+:count)
for ( auto e : data
      | std::ranges::views::drop(1) )
    count += e;
#pragma omp parallel for reduction(+:count)
for ( auto e : data
      | std::ranges::views::transform
      ( []( auto e ) { return 2*e; } ) )
    count += e;
```

C++ ranges speedup

```
==== Run range on 1 threads ====  
sum of vector: 50000005000000 in 6.148  
sum w/ drop 1: 50000004999999 in 6.017  
sum times 2 : 100000010000000 in 6.012  
==== Run range on 25 threads ====  
sum of vector: 50000005000000 in 0.494  
sum w/ drop 1: 50000004999999 in 0.477  
sum times 2 : 100000010000000 in 0.489  
==== Run range on 51 threads ====  
sum of vector: 50000005000000 in 0.257  
sum w/ drop 1: 50000004999999 in 0.248  
sum times 2 : 100000010000000 in 0.245  
==== Run range on 76 threads ====  
sum of vector: 50000005000000 in 0.182  
sum w/ drop 1: 50000004999999 in 0.184  
sum times 2 : 100000010000000 in 0.185  
==== Run range on 102 threads ====  
sum of vector: 50000005000000 in 0.143  
sum w/ drop 1: 50000004999999 in 0.139  
sum times 2 : 100000010000000 in 0.134  
==== Run range on 128 threads ====
```

```
sum of vector: 50000005000000 in 0.122  
sum w/ drop 1: 50000004999999 in 0.11
```

Ranges and indices

Use `iota_view` to obtain indices:

```
// iota.cxx
vector<long> data(N);
# pragma omp parallel for
for ( auto i : std::ranges::iota_view( 0UZ,data.size() ) )
    data[i] = f(i);
```

Note that this uses C++23 suffix for `unsigned size_t`. For older versions:

```
iota_view( static_cast<size_t>(0),data.size() )
```

Custom iterators, 0

Recall that

Short hand:

```
vector<float> v;  
for ( auto e : v )  
    ... e ...
```

for:

```
for ( vector<float>::iter  
      e=v.begin();  
      e!=v.end(); e++ )  
    ... *e ...
```

If we want

```
for ( auto e : my_object )  
    ... e ...
```

we need a sub-class for the iterator with methods such as *begin*, *end*, *** and *++*.

Probably also *+=* and *-*

Custom iterators, 1

OpenMP can parallelize any range-based loop with a random-access iterator.

Class:

```
// iterator.cxx
template<typename T>
class NewVector {
protected:
    T *storage;
    int s;
public:
    // iterator stuff
    class iter;
    iter begin();
    iter end();
};
```

Main:

```
NewVector<float> v(s);
#pragma omp parallel for
for ( auto e : v )
    cout << e << " ";
```

Custom iterators, 2

Required iterator methods:

```
NewVector<T>::iter& operator++();  
T& operator*();  
bool operator==( const NewVector::iter &other ) const;  
bool operator!=( const NewVector::iter &other ) const;  
// needed to OpenMP  
int operator-  
    ( const NewVector::iter& other ) const;  
NewVector<T>::iter& operator+=( int add );
```

This is a little short of a full random-access iterator; the difference depends on the OpenMP implementation.

Custom iterators, exercise

Write the missing iterator methods.

Here's something to get you started.

```
template<typename T>
class NewVector<T>::iter {
private: T *searcher;
};

template<typename T>
NewVector<T>::iter::iter( T* searcher )
    : searcher(searcher) {};

template<typename T>
NewVector<T>::iter NewVector<T>::begin() {
    return NewVector<T>::iter(storage); };

template<typename T>
NewVector<T>::iter NewVector<T>::end()    {
    return NewVector<T>::iter(storage+NewVector<T>::s); };
```

Custom iterators, solution

```
template<typename T>
bool NewVector<T>::iter::operator==
    ( const NewVector<T>::iter &other ) const {
    return searcher==other.searcher; };

template<typename T>
bool NewVector<T>::iter::operator!=
    ( const NewVector<T>::iter &other ) const {
    return searcher!=other.searcher; };

template<typename T>
NewVector<T>::iter& NewVector<T>::iter::operator++() {
    searcher++; return *this; };

template<typename T>
NewVector<T>::iter& NewVector<T>::iter::operator+=( int add )
{
    searcher += add; return *this; };
```

Custom iterators, solution

```
template<typename T>
T& NewVector<T>::iter::operator*() {
    return *searcher; };
// needed for OpenMP
template<typename T>
int NewVector<T>::iter::operator-
    ( const NewVector<T>::iter& other ) const {
    return searcher-other.searcher; };
```

OpenMP vs standard parallelism

Application: prime number marking (load unbalanced)

```
#pragma omp parallel for \  
    schedule(static)  
for ( int i=0; i<nsize; i++) {  
    results[i] =  
        one_if_prime( number(i) );  
}  
  
// primepolicy.cpp  
transform  
    ( std::execution::par,  
      numbers.begin(),numbers.end(),  
      results.begin(),  
      [] (int n) -> int {  
          return one_if_prime(n); }  
    );
```

Standard parallelism uses Threading Building Blocks (Intel) (TBB)
as backend

Timing

Threads: 1

TBB: Time: 392 msec

Stat: Time: 389 msec

Dyn: Time: 390 msec

Threads: 25

TBB: Time: 20 msec

Stat: Time: 32 msec

Dyn: Time: 17 msec

Threads: 51

TBB: Time: 13 msec

Stat: Time: 15 msec

Dyn: Time: 9 msec

Threads: 76

TBB: Time: 29 msec

Stat: Time: 11 msec

Dyn: Time: 6 msec

Threads: 102

TBB: Time: 61 msec

Stat: Time: 8 msec

Reductions vs standard parallelism

Application: prime number counting (load unbalanced)

```
#pragma omp parallel for \  
    schedule(guided,8) \  
    reduction(+:prime_count)  
for ( auto n : numbers ) {  
    prime_count += one_if_prime( n );  
}
```

```
// reducepolicy.cpp  
prime_count = transform_reduce  
    ( std::execution::par,  
      numbers.begin(),numbers.end(),  
      0,  
      std::plus<>{} ,  
      [] ( int n ) -> int {  
          return one_if_prime(n); }  
    );
```

Timing

Threads: 1

TBB: Time: 391 msec

Stat: Time: 390 msec

Dyn: Time: 389 msec

Threads: 25

TBB: Time: 20 msec

Stat: Time: 17 msec

Dyn: Time: 17 msec

Threads: 51

TBB: Time: 13 msec

Stat: Time: 9 msec

Dyn: Time: 8 msec

Threads: 76

TBB: Time: 14 msec

Stat: Time: 8 msec

Dyn: Time: 5 msec

Threads: 102

TBB: Time: 76 msec

Stat: Time: 5 msec

OpenMP reductions

Questions

1. Are simple reductions the same as in C?
2. Can you reduce `std::vector` like an array?
3. Precisely *what* can you reduce?
4. Any interesting examples?
5. Compare reductions to native C++ mechanisms.

Scalar reductions

Same as in C,
you can now use range syntax for the loop.

```
// range.cxx
#pragma omp parallel for reduction(+:count)
for ( auto e : data )
    count += e;
```

Reductions on vectors

Use the *data* method to extract the array on which to reduce. However, this does not work:

```
vector<float> x;  
#pragma omp parallel reduction(+:x.data())
```

because the reduction clause wants a variable, not an expression, for the array, so you need an extra bare pointer:

```
// reductarray.cxx  
vector<int> data(nthreads,0);  
int *datadata = data.data();  
#pragma omp parallel for schedule(static,1) \  
    reduction(+:datadata[:nthreads])
```

Reduction on class objects

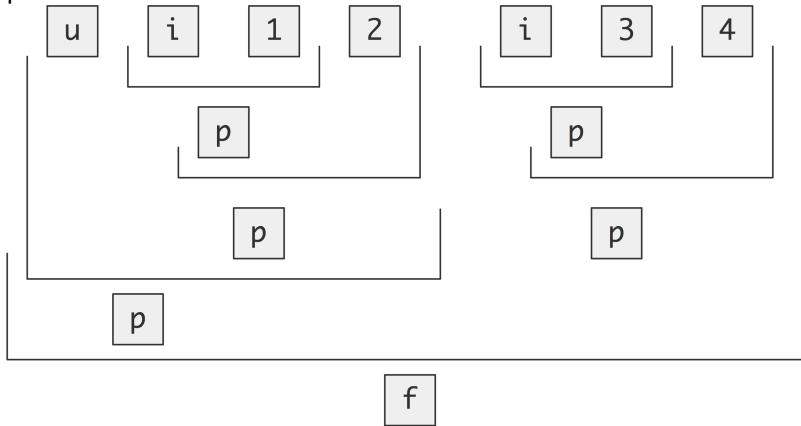
Reduction can be applied to any class for which the reduction operator is defined as `operator+` or whichever operator the case may be.

```
// reductclass.cxx
class Thing {
private:
    float x{0.f};
public:
    Thing() = default;
    Thing( float x ) : x(x) {};
    Thing operator+
        ( const Thing& other )
    {
        return Thing( x +
            other.x );
    };
};
```

```
vector< Thing >
    things(500,Thing(1.f) );
Thing result(0.f);
#pragma omp parallel for \
    reduction( +:result )
for ( const auto& t : things
    )
    result = result + t;
```

Reduction illustrated

Reduction of four items on two threads. `i` is the OpenMP initialization, and `u` is the user initialization; each `p` stands for a partial reduction value.



User-defined reductions, syntax

```
#pragma omp declare reduction  
( identifier : typelist : combiner )  
[initializer(initializer-expression)]
```

Reduction over iterators

Support for *C++ iterators*

```
#pragma omp declare reduction \  
    (merge                // identifier  
    : std::vector<int> // typelist  
    : omp_out.insert(omp_out.end(), omp_in.begin(),  
                      omp_in.end()) // combiner  
    )
```

Lambda expressions in declared reductions

You can use lambda expressions in the explicit expression for a declared reduction:

```
// reductexpr.cxx
#pragma omp declare reduction\
  (minabs : int : \
    omp_out = \
      [] (int x,int y) -> int { \
        return abs(x) > abs(y) ? abs(y) : abs(x); } \
    (omp_in,omp_out) ) \
  initializer (omp_priv=limit::max())
```

You can not assign the lambda expression to a variable and use that, because *omp_in/out* are the only variables allowed in the explicit expression.

Example category: histograms

Count which elements fall into what bin:

```
for ( auto e : some_range )  
    histogram[ value(e) ]++;
```

Collisions are possible, but unlikely, so critical section is very inefficient

Histogram: intended main program

Declare a reduction on a histogram object;
each thread gets a local map:

```
/*  
 * Reduction loop in main program  
 */  
bincounter<char> charcount;  
#pragma omp parallel for reduction(+ : charcount)  
for ( int i=0; i<text.size(); i++ )  
    charcount.inc( text[i] );
```

Q: why does the *inc* not have to be atomic?

Histogram solution: reduction operator

Give the class a += operator to do the combining:

```
// charcount.cxx
template<typename key>
class bincounter : public map<key,int> {
public:
// merge this with other map
void operator+=
    ( const bincounter<key>& other ) {
    for ( auto [k,v] : other )
        if ( map<key,int>::contains(k) )
            this->at(k) += v;
        else
            this->insert( {k,v} );
};
// insert one char in this map
void inc(char k) {
    if ( map<key,int>::contains(k) )
        this->at(k) += 1;
    else
        this->insert( {k,1} );
};
```

Histogram in native C++

Use atomics because there is no reduction mechanism:

```
// mapreduceatomic.cxx
class CharCounter : public array<atomic<int>,26> {
public:
    CharCounter() {
        for ( int ic=0; ic<26; ic++ )
            (*this)[ic] = 0;
    };
    // insert one char in this map
    void inc(char k) {
        if (k==' ') return;
        int ik = k-'a';
        (*this)[ik]++;
    };
};
```

Histogram in native C++, comparison

OpenMP reduction on `array<int,26>`:

<i>Using atomics</i>	<i>on 1 threads:</i>	<i>time= 20.19 msec</i>
<i>OpenMP reduction</i>	<i>on 1 threads:</i>	<i>time= 1.966 msec</i>
<i>Using atomics</i>	<i>on 5 threads:</i>	<i>time= 315.855 msec</i>
<i>OpenMP reduction</i>	<i>on 5 threads:</i>	<i>time= 0.52 msec</i>
<i>Using atomics</i>	<i>on 10 threads:</i>	<i>time= 91.968 msec</i>
<i>OpenMP reduction</i>	<i>on 10 threads:</i>	<i>time= 0.364 msec</i>
<i>Using atomics</i>	<i>on 30 threads:</i>	<i>time= 249.171 msec</i>
<i>OpenMP reduction</i>	<i>on 30 threads:</i>	<i>time= 0.556 msec</i>
<i>Using atomics</i>	<i>on 50 threads:</i>	<i>time= 164.177 msec</i>
<i>OpenMP reduction</i>	<i>on 50 threads:</i>	<i>time= 0.904 msec</i>

Exercise: mapreduce

Make an OpenMP parallel version of:

```
intcounter primecounter;  
for ( auto n : numbers )  
    if ( isprime(n) )  
        primecounter.add(n);
```

where *primecounter* contains a *map*<*int*,*int*>.

Use skeleton: `mapreduce.cxx`

Example category: list filtering

The sequential code is as follows:

```
vector<int> data(100);  
// fill the data  
vector<int> filtered;  
for ( auto e : data ) {  
    if ( f(e) )  
        filtered.push_back(e);  
}
```

List filtering, solution 1

Let each thread have a local array, and then to concatenate these:

```
#pragma omp parallel
{
    vector<int> local;
# pragma omp for
    for ( auto e : data )
        if ( f(e) ) local.push_back(e);
    filtered += local;
}
```

where we have used an append operation on vectors:

```
// filterreduct.cxx
template<typename T>
vector<T>& operator+=( vector<T>& me, const vector<T>& other
    ) {
    me.insert( me.end(), other.begin(), other.end() );
    return me;
};
```


List filtering, not quite solution 2

We could use the plus-is operation to declare a reduction:

```
#pragma omp declare reduction\  
(  
    \  
    +: vector<int>:omp_out += omp_in \  
    ) \  
    initializer( omp_priv = vector<int>{} )
```

Problem: OpenMP reductions can not be declared non-commutative, so the contributions from the threads may not appear in order.

Code:

```
#pragma omp parallel \  
    reduction(+ : filtered)  
{  
    vector<int> local;  
#   pragma omp for  
    for ( auto e : data )  
        if ( f(e) )  
            local.push_back(e);
```

Output:

```
Mod 5: 80 85 90 95  
    ↪ 100 5 10 15 20  
    ↪ 25 30 35 40 45  
    ↪ 50 55 60 65 70  
    ↪ 75
```

List filtering, task-based solution

Parallel region, without for:

Code:

```
// filtertask.cxx
vector<int> filtered;
int ithread=0;
#pragma omp parallel
{
    vector<int> local;
    int threadnum =
        omp_get_thread_num();
    # pragma omp for
    for ( auto e : data )
        if ( e%5==0 )
            local.push_back(e);
    // create task to add local
    to filtered
}
```

Output:

```
Mod 5: 5 10 15 20 25
      ↪ 30 35 40 45 50
      ↪ 55 60 65 70 75
      ↪ 80 85 90 95 100
```

List filtering, task-based solution'

The task spins until it's its turn:

Code:

```
# pragma omp task \  
    shared(filtered,ithread)  
{  
    // wait your turn  
    while  
        (threadnum>ithread) {  
    # pragma omp taskyield  
        }  
    // merge  
        filtered += local;  
        ithread++;  
    }
```

Output:

```
Mod 5: 5 10 15 20 25  
        ↪30 35 40 45 50  
        ↪55 60 65 70 75  
        ↪80 85 90 95 100
```

Templated reductions

You can reduce with a templated function if you put both the declaration and the reduction in the same templated function:

```
template<typename T>
T generic_reduction( vector<T> tdata ) {
#pragma omp declare reduction
    \
    (rwzt:T:omp_out=reduce_without_zero<T>(omp_out,omp_in))
    \
    initializer(omp_priv=-1.f)

    T tmin = -1;
#pragma omp parallel for reduction(rwzt:tmin)
    for (int id=0; id<tdata.size(); id++)
        tmin = reduce_without_zero<T>(tmin,tdata[id]);
    return tmin;
};
```

which is then called with specific data: